

## Quantum Machine Learning

**Topic 1 (*Expressivity of the generating set*).** Recently research has pointed to the encoding of data into a parameterised quantum circuit as being a key part of successful quantum machine learning, in this project you will attempt to characterise its importance.

In this project, you will assess the importance of the type, quantity and variety of encoding layers on the expressivity of parameterised quantum circuits, and study how different choices make parametrized quantum circuits (a.k.a. quantum neural networks) different from classical neural networks.

You will test this by using a parameterised quantum circuit with fixed input encoding <sup>1</sup> to generate a synthetic data set, then experiment with how expressive a parameterised quantum circuit must be to learn this data set. Repeating these experiments for multiple different architectures of the generating parameterised quantum circuits will allow for comparisons and orderings.

You may also experiment with how neural networks perform on the same data set, experimenting with different architectures to characterise how modifications to the generating architecture impact the need for more complicated neural networks.

Three possible choices of inquiry:

- Generators v. generators: understand the role of the spectrum of generators of rotations in the functions the circuits generate. Select two different classes of generators, and corresponding PQC-based models A and B. For a fixed number of qubits (2-3) Generate artificial data with A, learn with B and vice versa. study the quality of fit, and generalization performance, and how many parameters (depth) are needed for good fits (training error) (but what is the generalization/test error)? Discuss.
- Depth v. width: understand the importance of data-reuploading. Select two qubit numbers, say 1 and 3, and generate corresponding PQC-based models A and B with those qubit numbers, defined for the same data dimension (use data reuploading). Generate artificial data with A, learn with B and vice versa. study the quality of fit, and generalization performance, and how much depth is needed (in the shallower circuit) for good fits (training error) (but what is the generalization/test error)? Discuss.
- QNNs v. NNs: qualitative comparison of NN functions vs QNN functions. Consider two models A and B, where A is PQC-based, and B is a neural network, with the same dimension of inputs and outputs. Generate artificial data

---

<sup>1</sup>an input encoding is the scheme used to input data into a parameterised quantum circuit, it is normally composed of rotation gates that rotate the state by some amount dependent on the size of the input angle  $x_i$

with A, learn with B and vice versa. study the quality of fit, and generalization performance, and how parameter numbers and depth influence quality of fits (training error) (but what is the generalization/test error)? Discuss. Can NNs learn all QNNs can generate as efficiently and vice versa? What would you expect.

**Literature:** We have provided a number of sources to assist you. Papers 1 and 2 address how a quantum computer can be used for machine learning by using parameterised quantum circuits. Sources 3 and 4 are theoretical works that describe how different encoding layers may affect generalisation, these may provide inspiration for what types of encoding layers you may wish to study. Sources 5 and 6 are again practical implementations of quantum machine learning.

1. <https://arxiv.org/pdf/1804.11326.pdf>
2. <https://arxiv.org/pdf/1804.00633.pdf>
3. <https://journals.aps.org/pr/abstract/10.1103/PhysRevA.103.032430>
4. <https://arxiv.org/pdf/2106.03880.pdf>
5. <https://www.tensorflow.org/quantum/tutorials/mnist>
6. [https://pennyLane.ai/qml/demos/tutorial\\_variational\\_classifier.html](https://pennyLane.ai/qml/demos/tutorial_variational_classifier.html)

### Objectives and tasks:

- Generate artificial data from a quantum circuit by fixing the relevant hyperparameters and parameters.

**Remark.** *I.e., analogous to the artificial data in [arxiv.org/pdf/1804.11326.pdf](https://arxiv.org/pdf/1804.11326.pdf).*

- Experiment with the learning performance of different parameterized quantum circuits (or neural networks) on learning this data set, varying their architecture.
- Repeat this experimentation for different artificial datasets that originate from different hyperparameters.
- Compare the performance of several different encoding layers. How do changes to encoding affect training? How do changes in the test set affect this? Do the two effects cancel each other out?

### The report should contain:

1. An explanation of how the artificial data is generated.

2. An explanation of the code that generates the artificial data and trains a parameterized quantum circuit model/neural net to classify this data.
3. Comparison of different encoding schemes/architectures on the training task.

**For an 8+:** Experiment with using the encoding as a *regularization technique*.

**Topic 2 (*learning graph invariants with PQCs*).** Variational quantum algorithms are candidates for advantage on near-term quantum hardware. The choice of ansatz to solve a specific problem plays an important role in trainability and performance of the algorithm. So we are constantly looking for more informed ansatzes and the ansatz is used to define the learning model. One way of embedding bias, that is, prior knowledge that will enable your model to learn faster, in your model is through exploiting the symmetries of the problem. In this mini project, you are going to use a particular ansatz, an equivariant ansatz, in a data re-uploading scheme to learn a permutation invariant property of graphs, namely connectivity. A graph is said to be connected if there is a path between every pair of vertices. Then you are going to discuss the expressivity of the quantum model and compare its performance to a classical neural network.

**Literature:**

1. <https://arxiv.org/pdf/2210.07980.pdf>
2. <https://arxiv.org/pdf/2112.05261.pdf>

**Theory questions:**

Given an arbitrary matrix  $M \in U(N)$ , which in our case will be defined in next section and is in fact an equivariant encoding of a graph, and a sequence of matrices  $\{W_i\} \in U(N)$ , we define the  $k$ -th re-uploading circuit to be  $U_k = \prod_{i=1}^k (W_i M)$  and the function family realized by this circuit to be  $h_k(M) = \langle 0|U_k|0 \rangle^2$ .

1. By inserting resolutions of identity<sup>3</sup> matrix show that this function family is  $k$ -th degree multivariate polynomials of the entries of the matrix  $M$  of the form:

$$h_k(M) = \sum_{i_1=0}^{n-1} \cdots \sum_{i_k=0}^{n-1} m_{i_1} \times \cdots \times m_{i_k} \times w_{i_1}^{(1)} \cdots w_{i_k}^{(k)}$$

where  $i_l$  are multi-indices selecting an entry of a matrix,  $n = N^2$  and  $w_{i_l}^{(j)}$  are the  $i_l$  entry of the matrix  $W_j$ .

2. Show that there are  $n * k$  many parameters to set the coefficients  $w_{i_l}^{(j)}$  while having  $\binom{n+k-1}{k}$  many equations. what does this tell us about the expressivity of this model? Note that we would consider this model to be maximally expressive if one can get all the coefficients.

---

<sup>2</sup>To be precise, this  $h_k(M)$  is not fully defined but its absolute value is. However, we still use the former format without absolute values as it simplifies the construction in the next part.

<sup>3</sup> $\mathbb{I} = \sum_{i=0}^{2^m-1} |i\rangle\langle i|$  represents the identity operator in an  $m$ -qubit Hilbert space

3. Explain what a permutation equivariant ansatz is and how it is connected to graph symmetries. You can use the references to answer this question.

**Programming problems:** You will be using the data re-uploading quantum model with specific data encoding and variational layers to learn the connectivity problem. Then compare its performance to a classical model to draw your own conclusions about the expressivity of the quantum model.

**Data Generation:**

Use a graph generation algorithm to create a set of random graphs, specify the graphs by their adjacency matrix. To each adjacency matrix, assign a label by whether the graph is connected or disconnected. If the graph is connected, label it as 0; if it is disconnected, label it as 1. This will give you the training set  $T = \{(A_i, y_i)\}_{i=1}^N$  where  $A_i$  are the adjacency matrices and  $y_i = \{0, 1\}$ .

**Learning with a quantum and a classical model:**

1. Quantum model: The quantum circuit itself is constructed from  $L$  layers, each consisting of a data encoding circuit block  $M$  and a trainable circuit block  $W_i(\theta_i)$  controlled by the parameters  $\theta_i$ .
  - a- Let  $M = e^{iH_G}$  where  $H_G = \sum_{(i,j) \in \epsilon} \sigma_z^i \sigma_z^j$  is the Ising Hamiltonian  
For every adjacency matrix  $A$  of the graph  $G$  in the training data use the Ising Hamiltonian to encode the graphs into quantum circuits .
  - b- For variational parts of the circuit, take  $W_j(\theta_j) = \bigotimes_{i=1}^n R_x(\theta_j)$  where  $\theta_j$  is the trainable parameter in the layer j. Here we have one parameter to train per layer.
  - c- We can get more parameters by modifying the variational ansatz to  $W_j(\theta_j^{(1)}, \theta_j^{(2)}) = \bigotimes_{l=1}^n R_x(\theta_j^{(1)}) R_y(\theta_j^{(2)})$ . This helps with the expressivity of the model.
  - d- **Theory question:** Show that this circuit ansatz is permutation equivariant.
  - e- Train the model for different numbers of re-uploading layers for both variational ansatz. compare the performance.
  - f- You can also allow parametrization in the data encoding layers. allow  $M_j = e^{i\gamma_j H_G}$  where  $H_G = \sum_{(i,j) \in \epsilon} \sigma_z^i \sigma_z^j$  and  $\gamma_j$  is the trainable parameter in layer j. Compare the performance.
2. Classical model: Define a classical learning model where the hypothesis family are multivariate k degree polynomial functions:

$$h_k(M) = \sum_{i_1=0}^{n-1} \cdots \sum_{i_k=0}^{n-1} m_{i_1} \times \cdots \times m_{i_k} \times w_{i_1 \dots i_k}$$

where the coefficients  $w_{i_1 \dots i_k}$  are completely free. Train the model and optimize the parameters  $w_{i_1 \dots i_k}$  to learn connectivity of the graphs. Discuss the performance of the model and compare it to the quantum model with  $k$  layers.

**The report should contain:**

1. Answer to the questions in the theory part.
2. Report of the implementation of the quantum model with all the two suggested variational layers. Comparison of them to each other. Comparison of their performance to the case where you allow parametrization in the data encoding layers. Answers to the theory question (proving that the ansatz is permutation equivariant)
3. Report of the implementation of the classical model and its comparison to the quantum model.
4. **For grade 8+** We suggested  $W_j = \bigotimes_{l=1}^n R_x(\theta_j^{(1)})R_y(\theta_j^{(2)})$  for variational layers. This design, allows two parameters per qubit per layer to be trained. We can have three parameters by having  $W_j = \bigotimes_{l=1}^n R_x(\theta_j^{(1)})R_y(\theta_j^{(2)})R_z(\theta_j^{(3)})$ . Adding more parameters help with expressivity of the model. Design another equivariant variational ansatz with more trainable parameters in each layer to get better performance. Show that your ansatz is equivariant. Train the model and compare the performance.

**Topic 3 (*Towards learning separations*).** In this project you will use a quantum algorithm to solve a ML problem related to quantum many body physics. Specifically, the task is to learn to predict from data expectation values of unknown observables on time evolved quantum states under a known Hamiltonian. Alternatively, you can consider the task of learning to predict expectation values on ground states of a class of local Hamiltonians. As it will be discussed in the lectures, for a special class of Hamiltonians one could show that those tasks exhibit a provable learning separation with respect to classical algorithms. In this project, we will focus on a class of physically motivated Hamiltonians.

**Literature:**

1. <https://arxiv.org/pdf/2306.16028.pdf>
2. <https://www.science.org/doi/full/10.1126/science.abk3333>
3. <https://arxiv.org/pdf/2301.13169.pdf>
4. <https://arxiv.org/abs/1912.13146>
5. <https://arxiv.org/abs/2205.06278>

**Questions:**

- Understand and explain what we mean by separation in learning. Why is it different that a separation in computing? Can you give an example of a function which is hard to compute but not hard to learn?
- What are the assumptions on the class of Hamiltonians in the references 2-3? Explain, from an high level point of view, the algorithm in reference 3 and the main idea of why it is guaranteed to work.
- What are the main differences between the algorithms in references 2 and 3? (Resources, sample complexity, time complexity)

**Implementation:** There are two possible variants of the learning problem we can consider: in **case A** the task is to predict observables on time evolved quantum states, in **case B** we will be dealing with ground states instead. Notice that **case A** will be easier to implement as the quantum algorithm will just have to perform a fixed time evolution on input states. The implementation of **case B** will be instead more involved, as the quantum algorithm will have to find the ground state for the Hamiltonians associated at each data point. However, **case B** allows for a direct comparison with relevant recent results in the literature (Ref. 2-3) and may lead to interesting unexplored scenarios.

Anyway, you can choose either **case A** or **case B**. Notice that the final grade of the

miniproject will not be affected by the choice of **case A** or **case B**.

**Case A: Predicting unknown observables on time evolved states** The ML problem you will consider is of the same kind of the ones in Ref 2-3, however we consider the simpler case where we predict observables on time evolved states. Let  $H$  be a Hamiltonian and consider the associated concept class

$$\mathcal{F}_{H,O} = \{f^\alpha(x) \mid \alpha \in [-1, 1]^m\} \quad (1)$$

$$\text{with: } f^\alpha(x) : x \in \{0, 1\}^n \rightarrow f^\alpha(x) = \text{Tr}[\rho_H(x)O(\alpha)] \quad \& \quad O(\alpha) = \sum_{i=1}^m \alpha_i P_i \quad (2)$$

Where  $x$  specifies the initial state  $\rho_0(x) = |x\rangle\langle x|$  and  $\rho_H(x)$  is the evolved state under the Hamiltonian  $H$ , i.e.  $\rho_H(x) = U\rho_0(x)U^\dagger$  with  $U = e^{iHt}$ . The ML algorithm has to learn a model  $h(x)$  which approximates the function  $f^{\alpha^*}$  for an unknown  $\alpha^*$  using  $\mathcal{T}^{\alpha^*} = \{(x_\ell, y_\ell)\}_{\ell=1}^N$  as training data. In the ML problem you will consider, the input  $x$  will come from an underlying distribution. You can choose the uniform distribution over  $x \in \{0, 1\}^n$  for example.

1. We will consider the same class of Hamiltonian considered in Ref. 3 given by the 2D anti-ferromagnetic random Heisenberg model  $H(J) = \sum_{i,j} J_{ij}(X_i X_j + Y_i Y_j + Z_i Z_j)$ , you can set the coupling  $J_{ij}$  randomly. The Hamiltonian will act on a system of  $N = 10$  qubits.
2. As unknown observable you can choose an arbitrary combination of  $k$  local Pauli string  $O(\alpha) = \sum_i \alpha_i P_i$ . You can set  $k = 3$  for example. This means that the observable  $O(\alpha)$  will be the linear combination of all the 3-local Pauli matrices with an arbitrary (of your choice) but fixed  $\alpha$ .
3. At this point you can generate data classically by explicitly constructing the initial state  $\rho_0(x)$  and computing  $\rho_H(x)$  by matrix multiplication.
4. Construct the quantum model and train it using the LASSO regression algorithm:
  - (a) For every training point in  $\mathcal{T}^{\alpha^*} = \{(x_\ell, y_\ell)\}_{\ell=1}^N$  the quantum algorithm prepares multiple copies of the state  $\rho_H(x_\ell)$  and computes the estimates of the expectation values  $\langle P_j \rangle_\ell = \text{Tr}[\rho_H(x_\ell)P_j] \quad \forall j = 1, \dots, m$  up to a certain precision  $\epsilon_1$ . The number of copies needed will depend on the desired error  $\epsilon_1$  on the expectation values of each  $\langle P_j \rangle_\ell$ . As you saw in the lectures, you will have to take in consideration a sampling error which scales polynomially with the number of copies of the state.

Note, also, that  $m$  scales at most polynomially in  $n$  as  $\{P_j\}_{j=1}^m$  are local observables.

- (b) To prepare the states  $\rho_H(x_\ell)$  you can use Trotterization methods that you studied in the course. (You will have to choose an adequate number of Trotter steps!)
- (c) Define the model  $h(x) = w \cdot \phi(x)$ , where  $\phi(x)$  is the vector of the Pauli string expectation values  $\phi(x) = [\text{Tr}[\rho_H(x)P_1], \dots, \text{Tr}[\rho_H(x)P_m]]$  computed at Step 1. Set the right hyperparameter  $B \geq 0$  (motivate the choice!) and run the LASSO regression finds an optimal  $w^*$  from the following optimization process:

$$\min_{\substack{w \in \mathbb{R}^m \\ \|w\|_1 \leq B}} \frac{1}{N} \sum_{l=1}^N |w \cdot \phi(x_l) - y_l|^2 \quad (3)$$

with  $\{(x_l, y_l = \text{Tr}[\rho(x_l)O(\alpha)])\}_{l=1}^N$  being the training data.

Importantly, to meet the learning condition the optimization does not need to be solved exactly, i.e.  $w^* = \alpha$ . As reported in Ref. 3 it is sufficient to obtain a  $w^*$  whose training error is  $\epsilon_2$  larger than the optimal one.

- 5. Compare the results using a classical model (you can use your favourite Neural Network).

### The report should contain:

- 1. Answer to the questions above
- 2. Report of the ML experiment and comparison of the results between the classical and quantum method.
- 3. **8+** Try using different Hamiltonians and compare the results, specifically you should look at how the performance of the quantum model vs the classical one varies with respect to the Hamiltonian considered.
- 4. **9+** Discussion with formal arguments on why there should exist a Hamiltonian which gives rise to a learning separation for this kind of task. Identify an example of such Hamiltonian providing proofs of your claims.

**Case B (Hard): Predicting unknown observables on ground states**  
 The ML problem you will consider is of the same as the ones in Ref 2-3. Let

$\mathcal{H} = \{H(x) \mid x \in \{0,1\}^n\}$  be a family of local Hamiltonians and consider the associated concept class

$$\mathcal{F}_{\mathcal{H},O} = \{f^\alpha(x) \mid \alpha \in [-1,1]^m\} \quad (4)$$

$$\text{with: } f^\alpha(x) : x \in \{-1,1\}^n \rightarrow f^\alpha(x) = \text{Tr}[\rho_H(x)O(\alpha)] \quad \& \quad O(\alpha) = \sum_{i=1}^m \alpha_i P_i \quad (5)$$

Where  $x$  specifies the hamiltonian  $H(x)$  in the family  $\mathcal{H}$  and  $\rho_H(x)$  is the ground state of the Hamiltonian  $H(x)$ . The ML algorithm has to learn a model  $h(x)$  which approximates the function  $f^{\alpha^*}$  for an unknown  $\alpha^*$  using  $\mathcal{T}^{\alpha^*} = \{(x_\ell, y_\ell)\}_{\ell=1}^N$  as training data.

1. We will consider the same class of Hamiltonian considered in Ref. 2 given by the 2D anti-ferromagnetic random Heisenberg model  $H(J) = \sum_{i,j} J_{ij}(X_i X_j + Y_i Y_j + Z_i Z_j)$ , for  $N = 10$  qubits. Notice that in this case the couplings  $J_{ij}$  will exactly be your input vectors  $x$  for  $x \in \{-1,1\}^n$ . You may assume they come from a underlying distributions (e.g. uniform distribution).
2. As unknown observable you can choose an arbitrary combination of  $k$  local Pauli string  $O(\alpha_i) = \sum_i \alpha_i P_i$ . You can set  $k = 3$  for example. This means that the observable  $O(\alpha)$  will be the linear combination of all the 3-local Pauli matrices with an arbitrary (of your choice) but fixed  $\alpha$ .
3. At this point you can generate data classically by diagonalizing  $H(J)$  for different vectors  $J$  coming (as said before) from an underlying distribution. Notice that the theoretical guarantees for the classical ML algorithm of Ref.2 only works for data coming from the uniform distribution.
4. Construct the quantum model and train it using the LASSO regression algorithm:
  - (a) For every training point in  $T^\alpha = \{(x_\ell, y_\ell)\}_{\ell=1}^N$  the quantum algorithm prepares  $\text{poly}(n)$  copies of the state  $\rho_H(x_\ell)$  and computes the estimates of the expectation values  $\langle P_j \rangle_\ell = \text{Tr}[\rho_H(x_\ell) P_j] \quad \forall j = 1, \dots, m$  up to a certain precision  $\epsilon_1$ .  
Note that  $m$  scales at most polynomially in  $n$  as  $\{P_j\}_{j=1}^m$  are local observables.
  - (b) To prepare the states  $\rho_H(x_\ell)$  you can use the variational algorithm in Ref. 4-5
  - (c) Define the model  $h(x) = w \cdot \phi(x)$ , where  $\phi(x)$  is the vector of the Pauli string expectation values  $\phi(x) = [\text{Tr}[\rho_H(x) P_1], \dots, \text{Tr}[\rho_H(x) P_m]]$  computed at Step 1. Set the right hyperparameter  $B \geq 0$  (motivate the choice!)

and run the LASSO regression finds an optimal  $w^*$  from the following optimization process:

$$\min_{\substack{w \in \mathbb{R}^m \\ \|w\|_1 \leq B}} \frac{1}{N} \sum_{l=1}^N |w \cdot \phi(x_l) - y_l|^2 \quad (6)$$

with  $\{(x_l, y_l = \text{Tr}[\rho(x_l)O(\alpha)])\}_{l=1}^N$  being the training data.

Importantly, to meet the learning condition the optimization does not need to be solved exactly, i.e.  $w^* = \alpha$ . As reported in Ref. 3 it is sufficient to obtain a  $w^*$  whose training error is  $\epsilon_2$  larger than the optimal one.

5. Construct the classical model as described in Ref.3

**The report should contain:**

1. Answer to the questions above
2. Report of the ML experiment and comparison of the results between the classical and quantum method.
3. **8+** Try using different families of local Hamiltonians and compare the results, specifically you should look at how the performance of the quantum model vs the classical one varies with respect to the Hamiltonian family considered.
4. **9+** Discussion with formal arguments on why there should exist a family of Hamiltonians which give rise to a learning separation for this kind of task. Identify an example of such family of Hamiltonians providing proofs of your claim.